

Cryptography Notes By Engr. M Ahmad Nawaz**

** Cryptography Notes By Engr. M Ahmad Nawaz**

9-Week Complete Course with Advanced Mathematics, Step-by-Step Examples, and Real-World Scenarios

WEEK 1: OVERVIEW OF CRYPTOGRAPHY AND ITS EVOLUTION

Slide 1.1: Introduction to Cryptography

Definition

Cryptography is the practice and study of techniques for secure communication in the presence of third parties (adversaries). It constructs protocols that prevent third parties or the public from reading private messages.

Etymology

- **Kryptos** (Greek) = Hidden
- **Graphein** (Greek) = To write
- **Literal Meaning:** "Hidden Writing"

Core Objectives (CIA Triad + Non-Repudiation)

1. **Confidentiality:** Only authorized parties can access information.
 - *Mechanism:* Encryption (Symmetric/Asymmetric).
2. **Integrity:** Information cannot be altered undetectably.
 - *Mechanism:* Hash Functions, MACs, Digital Signatures.
3. **Availability:** Information is accessible when needed.
 - *Mechanism:* Redundancy, DDoS protection (often outside pure crypto).
4. **Non-Repudiation:** Sender cannot deny sending the message.
 - *Mechanism:* Digital Signatures.

Real-Time Scenario: Online Banking

- **Confidentiality:** Your password is encrypted via TLS (AES) before leaving your browser.

- **Integrity:** The transaction amount (\$100) is hashed; if changed to \$1000 in transit, the hash mismatch rejects the transaction.
- **Non-Repudiation:** A digital signature on the transfer request proves *you* authorized it, preventing you from claiming fraud later.

Slide 1.2: Historical Evolution of Cryptography

Timeline & Mathematical Shift

Era	Period	Cipher/Method	Key Feature	Mathematical Basis
Classical	1900 BC - 500 AD	Egyptian hieroglyphics, Atbash	Substitution	Simple Mapping
Classical	50-45 BC	Caesar Cipher	Shift cipher (k=3)	Modular Arithmetic ($\mathbb{Z}_{26}$)
Medieval	800-1200 AD	Al-Kindi's frequency analysis	Cryptanalysis	Statistics
Renaissance	1467	Alberti Cipher	Polyalphabetic	Multiple Alphabets
19th Century	1854	Playfair Cipher	Digraph substitution	Pairwise Mapping
WWI/WWII	1917-1945	Enigma Machine	Electromechanical rotor	Permutations & Reflections
Modern	1976	Diffie-Hellman	Public-key cryptography	Discrete Logarithm Problem
Modern	1977	RSA Algorithm	Asymmetric encryption	Integer Factorization
Contemporary	2001	AES	Symmetric block cipher	Substitution-Permutation Network (SPN)

Case Study: The Enigma Machine Break

- **Context:** Used by Nazi Germany in WWII.
- **Weakness:** A letter could never be encrypted as itself (reflector property).
- **Cryptanalysis:** Alan Turing and team at Bletchley Park used "Cribs" (known plaintext patterns like "Wetterbericht" - weather report) to reduce the key space from 10^{114} to solvable ranges using the "Bombe" machine.

Slide 1.3: Basic Mathematical Foundations - Sets & Groups

Set Theory Basics

Definition: A set is a well-defined collection of distinct objects.

Notation:

- **Set:** $S = \{a, b, c, d\}$
- **Element:** $a \in S$ (a belongs to S)
- **Empty Set:** $\emptyset = \{\}$
- **Cardinality:** $|S|$ = number of elements

Set Operations (Detailed Example)

Example: Let $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5, 6\}$, Universal Set $U = \{1..10\}$

1. **Union:** $A \cup B = \{1, 2, 3, 4, 5, 6\}$
2. **Intersection:** $A \cap B = \{3, 4\}$
3. **Difference:** $A - B = \{1, 2\}$ (Elements in A but not B)
4. **Complement:** $A' = U - A = \{5, 6, 7, 8, 9, 10\}$

Modular Arithmetic Sets ($\mathbb{Z}_n$)

Definition: $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$ (integers modulo n)

Operation: $a \bmod n$ is the remainder when a is divided by n .

Detailed Example: $\mathbb{Z}_{10}$

- $13 \bmod 10 = 3$ (Since $13 = 1 \times 10 + 3$)
- $-2 \bmod 10 = 8$ (Since $-2 = -1 \times 10 + 8$)
- **Addition:** $7 + 6 \bmod 10 = 13 \bmod 10 = 3$
- **Multiplication:** $7 \times 6 \bmod 10 = 42 \bmod 10 = 2$

Slide 1.4: Greatest Common Divisor (GCD) - Deep Dive

Definition

GCD(a, b) = largest integer d such that $d|a$ and $d|b$ (d divides a and b).

Euclidean Algorithm (Step-by-Step)

Theorem: $\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$

Detailed Example 1: GCD(252, 105)

Step 1: $252 = 105 \times 2 + 42 \rightarrow \text{GCD}(252, 105) = \text{GCD}(105, 42)$

Step 2: $105 = 42 \times 2 + 21 \rightarrow \text{GCD}(105, 42) = \text{GCD}(42, 21)$

Step 3: $42 = 21 \times 2 + 0 \quad \rightarrow \text{GCD}(42, 21) = \text{GCD}(21, 0) = 21$
 $\therefore \text{GCD}(252, 105) = 21$

Extended Euclidean Algorithm (Finding Modular Inverse)

Purpose: Find integers x, y such that: $ax + by = \text{GCD}(a, b)$.

This is crucial for finding $a^{-1} \pmod n$ in RSA.

Detailed Example: Find x, y for $252x + 105y = 21$

Backward Substitution:

1. From Step 2: $21 = 105 - 42 \times 2$
2. From Step 1: $42 = 252 - 105 \times 2$
3. Substitute (2) into (1):

$$21 = 105 - (252 - 105 \times 2) \times 2$$

$$21 = 105 - (252 \times 2 - 105 \times 4)$$

$$21 = 105 - 252 \times 2 + 105 \times 4$$

$$21 = 105 \times 5 - 252 \times 2$$

$$21 = 252 \times (-2) + 105 \times 5$$

Result: $x = -2, y = 5$

Application: To find $252^{-1} \pmod{105}$?

- Since GCD is 21 (not 1), the inverse does not exist.
- **Condition:** Inverse exists only if $\text{GCD}(a, n) = 1$.

Slide 1.5: Kirchhoff's Principle

Statement (1883)

"The security of a cryptosystem must depend **only** on the secrecy of the **key**, not on the secrecy of the **algorithm**."

Modern Interpretation & Real-World Scenario

Scenario: A company develops a proprietary encryption algorithm "SecureX" and keeps the code secret.

- **Risk:** If an employee leaks the code, the entire system is compromised forever.
- **Kirchhoff Approach:** Use AES (public algorithm). Even if the attacker knows you are using AES, they cannot decrypt without the Key.
- **Benefit:** If a key is compromised, you simply change the key, not the entire software infrastructure.

Kerckhoffs's Six Principles (Expanded)

- 1. **Unbreakable:** Practically impossible to decrypt without key.
- 2. **Compromise:** Losing the system details shouldn't inconvenience users (only key loss matters).
- 3. **Key Management:** Keys should be changeable and memorable (or storable securely).
- 4. **Telegraphy:** Applicable to digital communication channels.
- 5. **Portability:** Usable on mobile/embedded devices.
- 6. **Ease of Use:** No mental strain (automatic encryption).

Slide 1.6: Applications of Cryptography

Real-World Applications Table

Domain	Application	Cryptographic Technique	Real-Time Scenario
Web Security	HTTPS/TLS	AES, RSA, ECC	Buying a book on Amazon; credit card info encrypted.
Digital Payments	Credit cards, UPI	3DES, AES	Tapping card at store; terminal encrypts PIN.
Blockchain	Bitcoin, Ethereum	SHA-256, ECDSA	Mining involves finding hash below a target; signing transactions.
Messaging	WhatsApp, Signal	AES-256, Double Ratchet	End-to-end encryption; even server cannot read messages.
Authentication	Password hashing	bcrypt, Argon2	Database stores hash, not plain text password.
Digital Signatures	Document signing	RSA, DSA	Signing a PDF contract legally binding.
VPN	Secure tunneling	AES, ChaCha20	Working from home; corporate traffic encrypted over public WiFi.

Case Study: HTTPS Connection Handshake (Mathematical View)

Client	Server
--- ClientHello ----->	(Supports: TLS 1.3, AES-GCM, X25519)
<-- ServerHello -----	(Selects: TLS 1.3, AES-GCM, X25519)
<-- Certificate -----	(Contains Server Public Key)
--- Key Exchange ----->	(Client sends Encrypted Pre-Shared Key)

```
|<-- ServerHelloDone -----|
|                               |
|--- ChangeCipherSpec --->| (Switch to Encrypted Channel)
|--- Finished ----->| (Verify Handshake Integrity)
|                               |
|<-- ChangeCipherSpec -----|
|<-- Finished -----|
|                               |
|==== Encrypted Data =====| (Symmetric AES Key now active)
```

Mathematical Note: The "Key Exchange" often uses ECDH (Elliptic Curve Diffie-Hellman). Both parties generate a shared secret S without transmitting S directly.

$$S = \text{Private}_{client} \times \text{Public}_{server} = \text{Private}_{server} \times \text{Public}_{client}$$

WEEK 2: SYMMETRIC KEY ALGORITHMS

Slide 2.1: Symmetric Key Cryptography

Definition

Symmetric Key Cryptography: Same key used for both encryption and decryption. Also called Secret-Key Cryptography.

Mathematical Model

Let:

- **P** = Plaintext space
- **C** = Ciphertext space
- **K** = Key space
- **E**: $K \times P \rightarrow C$ (Encryption function)
- **D**: $K \times C \rightarrow P$ (Decryption function)

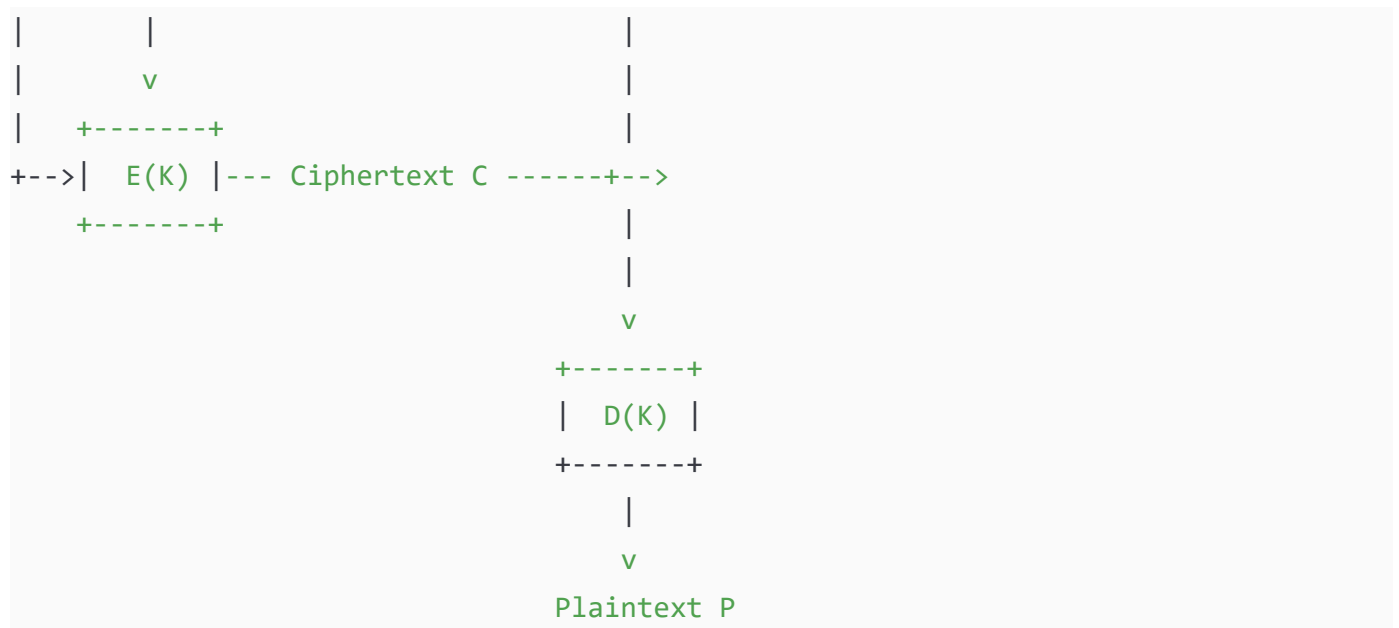
Encryption: $C = E(K, P)$

Decryption: $P = D(K, C)$

Correctness Condition: $\forall K \in \mathcal{K}, \forall P \in \mathcal{P} : D(K, E(K, P)) = P$

Diagram: Symmetric Key System





Real-Time Scenario: File Encryption

- **Tool:** VeraCrypt or BitLocker.
- **Process:** You set a password. A Key Derivation Function (KDF) turns the password into a 256-bit AES key. The hard drive sector is encrypted with this key. Without the password, the data looks like random noise.

Slide 2.2: Caesar Cipher - Mathematical Formulation

Historical Background

- Used by Julius Caesar (58-50 BC).
- Shift of **k = 3** positions.

Mathematical Formulation

Alphabet: $\mathbb{Z}_{26} = \{0, 1, 2, \dots, 25\}$ where A=0, B=1, ..., Z=25.

Encryption:

$$C = E(k, P) = (P + k) \mod 26$$

Decryption:

$$P = D(k, C) = (C - k) \mod 26$$

Note: In modular arithmetic, $(C - k) \mod 26$ is equivalent to $(C + (26 - k)) \mod 26$ to avoid negative numbers.

Detailed Example 1: Encryption with k=3

Plaintext: "HELLO"

Mapping: H=7, E=4, L=11, L=11, O=14

Step-by-step:

- 1. **H (7)**: $(7 + 3) \bmod 26 = 10 \rightarrow \text{K}$
 - 2. **E (4)**: $(4 + 3) \bmod 26 = 7 \rightarrow \text{H}$
 - 3. **L (11)**: $(11 + 3) \bmod 26 = 14 \rightarrow \text{O}$
 - 4. **L (11)**: $(11 + 3) \bmod 26 = 14 \rightarrow \text{O}$
 - 5. **O (14)**: $(14 + 3) \bmod 26 = 17 \rightarrow \text{R}$
- Ciphertext: "KHOOR"

Detailed Example 2: Decryption with k=3

Ciphertext: "KHOOR"
Mapping: K=10, H=7, O=14, O=14, R=17
Step-by-step:

- 1. **K (10)**: $(10 - 3) \bmod 26 = 7 \rightarrow \text{H}$
 - 2. **H (7)**: $(7 - 3) \bmod 26 = 4 \rightarrow \text{E}$
 - 3. **O (14)**: $(14 - 3) \bmod 26 = 11 \rightarrow \text{L}$
 - 4. **O (14)**: $(14 - 3) \bmod 26 = 11 \rightarrow \text{L}$
 - 5. **R (17)**: $(17 - 3) \bmod 26 = 14 \rightarrow \text{O}$
- Plaintext: "HELLO"

Edge Case Example: Wrapping Around

Plaintext: "XYZ", Key: $k = 3$

- 1. **X (23)**: $(23 + 3) = 26 \equiv 0 \bmod 26 \rightarrow \text{A}$
 - 2. **Y (24)**: $(24 + 3) = 27 \equiv 1 \bmod 26 \rightarrow \text{B}$
 - 3. **Z (25)**: $(25 + 3) = 28 \equiv 2 \bmod 26 \rightarrow \text{C}$
- Ciphertext: "ABC"

Slide 2.3: Caesar Cipher - Cryptanalysis

Brute Force Attack

Key Space: Only 25 possible keys ($k = 1, 2, \dots, 25$). $k = 0$ is identity.
Attack Method: Try all 25 shifts.

Detailed Example: Breaking "KHOOR"

Key	Calculation (C - k)	Decryption
k=1	10-1=9 (J)	JGNNQ
k=2	10-2=8 (I)	IFMMP
k=3	10-3=7 (H)	HELLO ✓ (Meaningful!)
k=4	10-4=6 (G)	GDKKN

Time Complexity: $O(|\mathcal{K}|)$. Since $|\mathcal{K}| = 25$, this is instant.

Frequency Analysis (Statistical Attack)

English Letter Frequencies (approximate):

- **E:** 12.7% (Most Common)
- **T:** 9.1%
- **A:** 8.2%
- **O:** 7.5%

Attack Strategy:

1. Count frequency of letters in ciphertext.
2. Assume the most frequent ciphertext letter corresponds to 'E'.
3. Calculate shift: $k = (\text{CipherLetter} - \text{'E'}) \bmod 26$.

Detailed Example: Frequency Analysis

Ciphertext: "KHOOR ZRUOG" (Length 11)

Frequency Count:

- O: 3 times (27%)
- R: 2 times
- K, H, Z, U, G: 1 time each

Hypothesis 1: 'O' (14) corresponds to 'E' (4).

- $k = (14 - 4) \bmod 26 = 10$
- Decrypt "KHOOR" with $k = 10$:
 - $K(10) - 10 = 0$ (A)
 - $H(7) - 10 = -3 = 23$ (X)
 - Result: "AX..." (Not English)

Hypothesis 2: 'O' (14) corresponds to 'T' (19).

- $k = (14 - 19) \bmod 26 = -5 = 21$
- Decrypt "KHOOR" with $k = 21$:
 - $K(10) - 21 = -11 = 15$ (P)
 - Result: "PM..." (Not English)

Hypothesis 3: 'O' (14) corresponds to 'O' (14). (Shift 0? No, looks shifted).

Let's try shifting 'O' to common vowels.

Actually, look at pattern "KHOOR". Double 'O'. In English, double letters are often 'LL', 'EE', 'SS'.
If 'O' = 'L' (11):

- $k = 14 - 11 = 3$.
 - Decrypt with $k = 3$: "HELLO WORLD" ✓
-

Slide 2.4: Generalized Shift & Affine Cipher

Security Analysis of Shift Cipher

Vulnerabilities:

1. **Small Key Space:** $|\mathcal{K}| = 25$.
2. **Preserves Frequency:** Distribution of letters remains identical, just shifted.
3. **Deterministic:** Same plaintext always yields same ciphertext.

Improvement: Affine Cipher

Mathematical Definition:

Encryption:

$$C = E(a, b, P) = (a \cdot P + b) \mod 26$$

Decryption:

$$P = D(a, b, C) = a^{-1} \cdot (C - b) \mod 26$$

Constraints:

- $\text{GCD}(a, 26) = 1$ (a must be coprime to 26 to have an inverse).
- Valid values of a : $\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$ (12 values).
- Valid values of b : $\{0, \dots, 25\}$ (26 values).
- **Key Space:** $12 \times 26 = 312$ keys.

Detailed Solved Example: Affine Cipher

Key: $a = 5, b = 8$.

Check Coprime: $\text{GCD}(5, 26) = 1$. OK.

Find Inverse a^{-1} :

- Need $5 \cdot x \equiv 1 \mod 26$.
- Trial: $5 \times 5 = 25 \equiv -1$. So $5 \times (-5) \equiv 1$. $-5 \equiv 21$.
- Check: $5 \times 21 = 105 = 4 \times 26 + 1$. Yes.
- $a^{-1} = 21$.

Encrypt Plaintext "HI":

- $H = 7, I = 8$.
- **H:** $C = (5 \cdot 7 + 8) \bmod 26 = (35 + 8) \bmod 26 = 43 \bmod 26 = 17 \rightarrow R$
- **I:** $C = (5 \cdot 8 + 8) \bmod 26 = (40 + 8) \bmod 26 = 48 \bmod 26 = 22 \rightarrow W$
- **Ciphertext:** "RW"

Decrypt "RW":

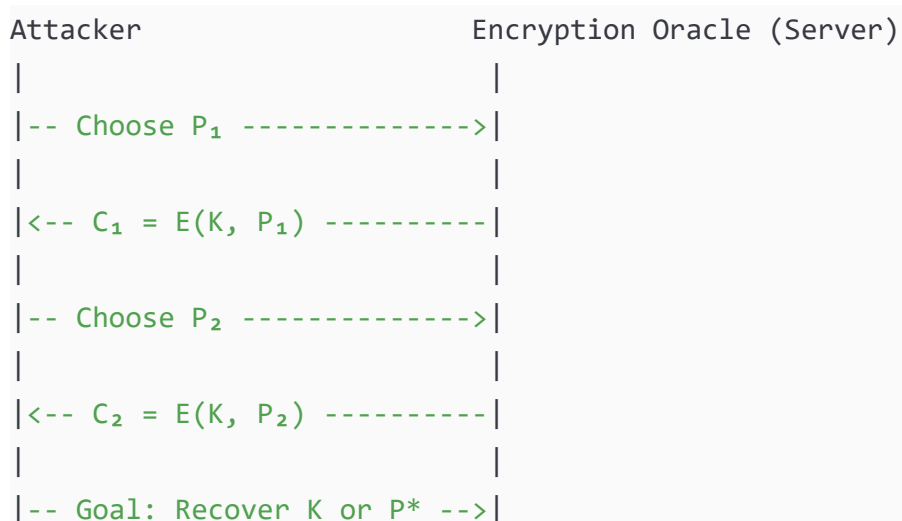
- $R = 17, W = 22$.
- Formula: $P = 21 \cdot (C - 8) \bmod 26$.
- **R:** $P = 21 \cdot (17 - 8) = 21 \cdot 9 = 189$.
 - $189 \div 26 = 7 \text{ rem } 7$.
 - $P = 7 \rightarrow H$.
- **W:** $P = 21 \cdot (22 - 8) = 21 \cdot 14 = 294$.
 - $294 \div 26 = 11 \text{ rem } 8$.
 - $P = 8 \rightarrow I$.
- **Plaintext:** "HI" ✓

Slide 2.5: Chosen Plaintext Attack (CPA)

Definition

Chosen Plaintext Attack (CPA): Attacker can obtain ciphertexts for plaintexts of their choice. This models a scenario where an attacker can trick a system into encrypting specific data.

Attack Model Diagram



CPA on Caesar Cipher (Trivial Break)

Attack Strategy:

1. Choose $P = "A"$ (value 0).

2. Obtain $C = E(k, "A")$.
3. Calculate $k = (C - 0) \bmod 26 = C$.

Example:

- Attacker sends "A".
- Oracle returns "D" (value 3).
- Key $k = 3$.
- **Result:** Key recovered in 1 query.

Security Definition: CPA-Security

Formal Definition: A cipher is CPA-secure if no polynomial-time adversary can distinguish between encryptions of two chosen plaintexts (M_0, M_1) with probability significantly better than $1/2$.

- **Caesar Cipher:** ✗ NOT CPA-secure (Deterministic).
 - **Modern Requirement:** Must be probabilistic (use IV/Nonce) to be CPA-secure.
-

Slide 2.6: Chosen Ciphertext Attack (CCA)

Definition

Chosen Ciphertext Attack (CCA): Attacker can obtain decryptions of ciphertexts of their choice (except the target ciphertext). This models scenarios where an attacker can send malformed data to a server and observe error messages.

CCA on Caesar Cipher

Attack Strategy:

1. Choose any ciphertext C .
2. Obtain $P = D(k, C)$.
3. Calculate $k = (C - P) \bmod 26$.

Example:

- Attacker sends $C = "G"$ (6).
- Oracle returns $P = "D"$ (3).
- $k = 6 - 3 = 3$.

Real-World CCA: Padding Oracle Attack

Scenario: CBC mode with PKCS#7 padding.

Vulnerability: Server returns different error codes for "Invalid Padding" vs "Decryption Failed".

Attack Process:

1. Attacker modifies the last byte of a ciphertext block.
2. Sends to server.

3. If server says "Invalid Padding", the byte guess was wrong.
4. If server says "Decryption Failed" (but padding ok), the byte guess was correct.
5. Attacker iterates byte-by-byte to decrypt the message without the key.

Impact: Broke SSL/TLS implementations (POODLE, Lucky13 attacks).

Mitigation: Use "Encrypt-then-MAC" or AEAD modes (like GCM).

WEEK 3: INTRODUCTION TO STREAM CIPHERS

Slide 3.1: Stream Cipher Fundamentals

Definition

Stream Cipher: Encrypts plaintext one bit/byte at a time by combining with a pseudorandom keystream. Ideal for hardware or where data arrives in streams (e.g., audio/video).

Mathematical Model

Let:

- $\mathbf{P} = P_1P_2P_3\dots$ (plaintext stream)
- $\mathbf{K} = K_1K_2K_3\dots$ (keystream)
- $\mathbf{C} = C_1C_2C_3\dots$ (ciphertext stream)

Encryption:

$$C_i = P_i \oplus K_i$$

Decryption:

$$P_i = C_i \oplus K_i$$

where $\oplus$ is the XOR operation.

XOR Properties (Crucial for Crypto)

1. **Commutative:** $A \oplus B = B \oplus A$
2. **Associative:** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
3. **Identity:** $A \oplus 0 = A$
4. **Self-inverse:** $A \oplus A = 0$
5. **Cancellation:** $A \oplus B \oplus B = A \oplus 0 = A$

Detailed Binary Example

Plaintext Byte: 01011010 (0x5A)

Keystream Byte: 11001101 (0xCD)

Encryption:

```
01011010
⊕ 11001101
-----
10010111 (0x97)
```

Decryption:

```
10010111
⊕ 11001101
-----
01011010 (Original Plaintext)
```

Slide 3.2: One-Time Pad (OTP)

Definition

One-Time Pad: Stream cipher where keystream is **truly random**, **as long as plaintext**, and **never reused**.

Properties

1. **Perfect Secrecy** (Shannon, 1949):

$$P(M|C) = P(M)$$

Ciphertext reveals **no information** about plaintext. Even with infinite computing power, it cannot be broken.

2. **Key Requirements:**

- Key must be **truly random** (not algorithmic).
- Key must be **at least as long** as plaintext.
- Key must **never be reused**.
- Key must be **kept secret**.

Mathematical Proof of Perfect Secrecy

Theorem: OTP provides perfect secrecy.

Proof:

For any plaintext M and ciphertext C of length n :

KaTeX parse error: Can't use function '\$' in math mode at position 23: ...= \frac{1}{2^n}\$\$\$ (Each ciphert...

Since $P(C) = \sum P(C|M') \cdot P(M') = \frac{1}{2^n} \cdot \sum P(M') = \frac{1}{2^n}$

$$P(M|C) = \frac{(1/2^n) \cdot P(M)}{1/2^n} = P(M)$$

∴ Knowing C does not change the probability of M . ✓

Detailed Example: OTP Encryption

Plaintext: $M = 10110011$ (binary)

Key: $K = 01101001$ (random, same length)

Encryption:

```

10110011
⊕ 01101001
-----
11011010 (Ciphertext)

```

Decryption:

```

11011010
⊕ 01101001
-----
10110011 (Plaintext)

```

Why OTP is Impractical (The Key Distribution Problem)

Problems:

1. **Key Distribution:** How to securely share a 1GB key to encrypt a 1GB file? If you have a secure channel for the key, why not send the file there?
2. **Key Storage:** Must store key as long as all messages combined.
3. **Key Generation:** True randomness requires hardware (noise diodes), not software.
4. **No Reuse:** Reusing key breaks security completely.

Key Reuse Attack (Two-Time Pad)

Scenario: Same keystream K used for two messages P_1, P_2 .

$$C_1 = P_1 \oplus K$$

$$C_2 = P_2 \oplus K$$

Attacker computes:

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2 \oplus (K \oplus K) = P_1 \oplus P_2$$

Result: Attacker gets XOR of two plaintexts.

Cryptanalysis: Since English has structure (spaces are 0x20, letters are 0x41-0x5A), statistical

analysis on $P_1 \oplus P_2$ can recover both messages.

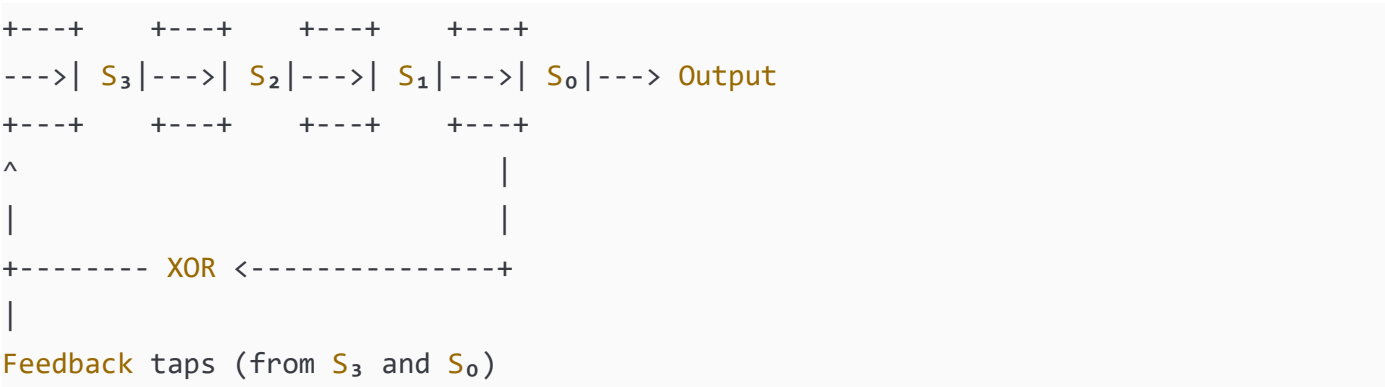
Real-World Example: Venona Project (US decrypted Soviet spies by recovering reused OTP pages).

Slide 3.3: Keystream Generators (LFSR)

Linear Feedback Shift Register (LFSR)

Definition: Shift register whose input bit is a linear function (XOR) of its previous state. Used in hardware for fast keystream generation.

LFSR Structure (4-bit Example)



Mathematical Representation

For an n-bit LFSR with feedback polynomial:

$$f(x) = c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

Recurrence relation:

$$s_{t+n} = c_{n-1} s_{t+n-1} \oplus c_{n-2} s_{t+n-2} \oplus \dots \oplus c_0 s_t$$

Detailed Example: 4-bit LFSR

Feedback polynomial: $f(x) = x^4 + x + 1$ (Taps at positions 4 and 1).

Initial State: $[S_3, S_2, S_1, S_0] = [1, 0, 0, 1]$

Feedback Bit: $S_3 \oplus S_0$

Step	State (S3 S2 S1 S0)	Feedback (S3 $\oplus$ S0)	Output (S0)
0	1 0 0 1	1 $\oplus$ 1 = 0	1
1	0 1 0 0	0 $\oplus$ 0 = 0	0
2	0 0 1 0	0 $\oplus$ 0 = 0	0
3	0 0 0 1	0 $\oplus$ 1 = 1	1
4	1 0 0 0	1 $\oplus$ 0 = 1	0

Step	State (S3 S2 S1 S0)	Feedback (S3 $\oplus$ S0)	Output (S0)
5	1 1 0 0	1 $\oplus$ 0 = 1	0
6	1 1 1 0	1 $\oplus$ 0 = 1	0
7	1 1 1 1	1 $\oplus$ 1 = 0	1
...	...	...	...

Period: Maximum period for n-bit LFSR is $2^n - 1$.

For 4-bit: $2^4 - 1 = 15$. (State `0000` is forbidden as it locks the register).

Maximum Length LFSR

Theorem: An n-bit LFSR has maximum period $2^n - 1$ if and only if its feedback polynomial is **primitive**.

Primitive Polynomials (examples):

- n=4: $x^4 + x + 1$
- n=8: $x^8 + x^4 + x^3 + x^2 + 1$
- n=16: $x^{16} + x^{12} + x^3 + x + 1$

Slide 3.4: Security Analysis of Stream Ciphers

Attacks on Stream Ciphers

1. Known Plaintext Attack

If attacker knows (P, C) pair:

$$K = P \oplus C$$

Keystream recovered immediately!

Implication: Never encrypt known headers (like "From:") with stream ciphers without additional protection.

2. Keystream Reuse Attack

Real-world example: WEP (Wired Equivalent Privacy).

- **Flaw:** RC4 keystream reused with small IVs.
- **Attack:** Attacker collects packets, finds IV collisions, computes $P_1 \oplus P_2$, recovers keys.
- **Result:** WEP is completely broken; replaced by WPA2/WPA3.

3. Correlation Attack

Idea: Exploit statistical correlation between LFSR output and keystream.

Example: Combining generator with Boolean function f .

If f has correlation with one LFSR:

$$P(f(x_1, x_2, \dots, x_n) = x_1) = p \neq 1/2$$

Attacker can recover x_1 faster than brute force.

Security Requirements

For a secure stream cipher:

- 1. **Long period:** Period $\geq 2^{128}$.
- 2. **Statistical randomness:** Pass all NIST statistical tests (Frequency, Runs, etc.).
- 3. **Large key size:** $|K| \geq 128$ bits.
- 4. **No correlations:** Output uncorrelated with key/IV.
- 5. **Fast:** Efficient in hardware/software.

Real Stream Ciphers Comparison

Cipher	Key Size	Status	Notes
RC4	40-2048 bits	X Broken	Biases in keystream (first bytes).
A5/1	64 bits	X Broken	GSM encryption; rainbow tables exist.
Salsa20	256 bits	✓ Secure	eSTREAM finalist; software optimized.
ChaCha20	256 bits	✓ Secure	Improved Salsa20; Used in TLS 1.3, SSH, WireGuard.

Slide 3.5: RC4 Stream Cipher (Deep Dive)

Overview

- Designed by Ron Rivest (1987).
- Used in SSL/TLS, WEP, WPA.
- Now **deprecated** due to vulnerabilities.

RC4 Algorithm (Step-by-Step)

Key Scheduling Algorithm (KSA): Initializes the state array S .

```
def KSA(key):
    S = [i for i in range(256)]
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
```

```

        S[i], S[j] = S[j], S[i] # swap
    return S

```

Pseudo-Random Generation Algorithm (PRGA): Generates keystream.

```

def PRGA(S, length):
    i = 0
    j = 0
    keystream = []
    for _ in range(length):
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i] # swap
        K = S[(S[i] + S[j]) % 256]
        keystream.append(K)
    return keystream

```

Detailed KSA Example (First 3 steps)

Key: "KEY" -> [75, 69, 89] (ASCII)

Initial S: [0, 1, 2, ..., 255]

Iteration i=0:

- $j = (0 + S[0] + key[0]) \% 256 = (0 + 0 + 75) \% 256 = 75$
- Swap $S[0]$ and $S[75]$.
- S becomes: [75, 1, 2, ..., 0, ..., 255]

Iteration i=1:

- $j = (75 + S[1] + key[1]) \% 256 = (75 + 1 + 69) \% 256 = 145$
- Swap $S[1]$ and $S[145]$.

Iteration i=2:

- $j = (145 + S[2] + key[2]) \% 256 = (145 + 2 + 89) \% 256 = 236$
- Swap $S[2]$ and $S[236]$.

RC4 Vulnerabilities

1. Key Recovery Attack (Fluhrer, Mantin, Shamir 2001)

- **Weak keys:** Certain IVs leak key bytes.
- **Attack complexity:** $O(2^{30})$ instead of $O(2^{128})$.
- **Impact:** Broke WEP.

2. Biases in Keystream

- **First byte bias:** $P(S[0] = 0) \approx 1/256 + 1/256^2$.
- **Second byte bias:** $P(S[1] = 0) \approx 1/256 - 1/256^2$.
- **Implication:** After collecting enough traffic, statistical anomalies reveal plaintext.

3. RC4 in TLS (Bar Mitzvah Attack, 2015)

- **Attack:** Collect 2^{26} encryptions of same plaintext.
- **Result:** Recover plaintext with high probability.
- **Response:** RFC 7465 prohibits RC4 in TLS.

Modern Alternative: ChaCha20

Advantages over RC4:

- ✓ No known biases.
- ✓ Faster in software (uses ARX: Add-Rotate-XOR).
- ✓ Simpler design.
- ✓ Used in TLS 1.3, SSH, WireGuard.

WEEK 4: INTRODUCTION TO BLOCK CIPHERS

Slide 4.1: Block Cipher Fundamentals

Definition

Block Cipher: Encrypts fixed-size blocks of plaintext to ciphertext blocks of same size.

- Unlike stream ciphers, block ciphers process data in chunks (e.g., 128 bits).

Mathematical Model

Let:

- n = Block size (bits)
- k = Key size (bits)
- $\mathbf{P} \in \{0, 1\}^n$ = Plaintext block
- $\mathbf{C} \in \{0, 1\}^n$ = Ciphertext block
- $\mathbf{K} \in \{0, 1\}^k$ = Key

Encryption:

$$C = E(K, P)$$

Decryption:

$$P = D(K, C) = E^{-1}(K, C)$$

Properties

1. **Invertibility:** $D(K, E(K, P)) = P$.

2. **Deterministic:** Same $(K, P) \rightarrow$ same C .
3. **Pseudorandom Permutation (PRP):** To an observer without K , $E(K, \cdot)$ looks like a random shuffle of all possible blocks.

Common Block Sizes

Cipher	Block Size	Key Size	Status
DES	64 bits	56 bits	Deprecated
3DES	64 bits	112/168 bits	Deprecated
AES	128 bits	128/192/256 bits	Standard
IDEA	64 bits	128 bits	Legacy
Blowfish	64 bits	32-448 bits	Legacy

Slide 4.2: Block Cipher Modes of Operation - ECB

Electronic Codebook (ECB) Mode

Concept: Each block encrypted independently.

$$C_i = E(K, P_i)$$

Diagram: ECB Mode

Encryption:

P_1 ----> $[E(K)]$ ----> C_1

P_2 ----> $[E(K)]$ ----> C_2

P_3 ----> $[E(K)]$ ----> C_3

Detailed Example

Plaintext: "HELLO WORLD!!!" (16 bytes)

Block size: 8 bytes

Key: K

$P_1 = \text{"HELLO WO"} \rightarrow C_1 = E(K, \text{"HELLO WO"})$

$P_2 = \text{"RLD!!!" + padding} \rightarrow C_2 = E(K, \text{"RLD!!!\x03\x03\x03"})$

Advantages

- ✓ Simple.
- ✓ Parallel encryption/decryption.
- ✓ No error propagation (error in C_1 only affects P_1).

Disadvantages (Critical)

- ✗ **Identical plaintext blocks** → **identical ciphertext blocks**.
- ✗ **Patterns visible**.
- ✗ **Not semantically secure**.

Famous Example: ECB Penguin

- **Scenario:** Encrypting a bitmap image of a penguin (black/white pixels).
- **Result:** When encrypting with ECB, the ciphertext image still shows the outline of the penguin.
- **Reason:** All white pixels (same plaintext block) encrypt to the same ciphertext block (same color).
- **Lesson:** Never use ECB for structured data.

Slide 4.3: Block Cipher Modes - CBC

Cipher Block Chaining (CBC) Mode

Concept: Each plaintext block XORed with previous ciphertext block before encryption. Introduces dependency.

Mathematical Formulation

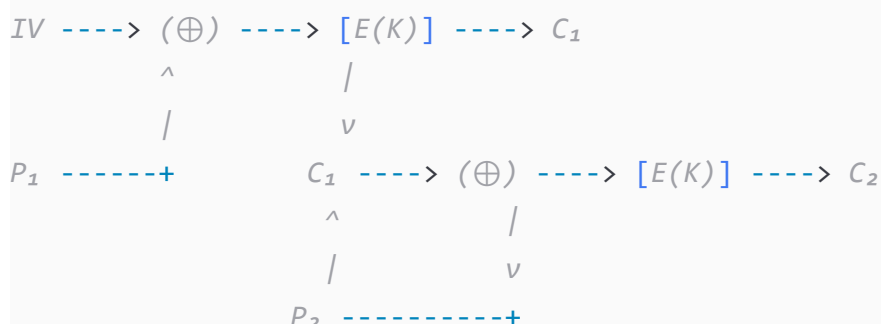
KaTeX parse error: Can't use function '\$' in math mode at position 9: C_0 = IV\$\$\$ (Initializati...

Decryption:

$$P_i = D(K, C_i) \oplus C_{i-1}$$

Diagram: CBC Mode

Encryption:



Detailed Example

Plaintext: $P = P_1 || P_2$

IV: Random 128-bit value

Step 1: $X_1 = P_1 \oplus IV$

Step 2: $C_1 = E(K, X_1)$

Step 3: $X_2 = P_2 \oplus C_1$

Step 4: $C_2 = E(K, X_2)$

Ciphertext: $IV || C_1 || C_2$ (IV must be sent with ciphertext).

Advantages

- ✓ **Hides patterns:** Same P_1 with different IV $\rightarrow$ different C .
- ✓ **Semantic security** (with random IV).
- ✓ Widely used (SSL/TLS, IPsec).

Disadvantages

- ✗ **Sequential encryption** (cannot parallelize encryption).
- ✓ **Parallel decryption** possible.
- ✗ **Error propagation:** One bit error in C_i affects P_i (garbled) and P_{i+1} (bit flip).
- ✗ **Padding oracle attacks** possible.

Padding in CBC (PKCS#7)

Rule: If block size = B bytes, and last block has L bytes, add $(B - L)$ bytes, each with value $(B - L)$.

Example: Block size = 8 bytes.

- Need 3 bytes padding $\rightarrow$ add `\x03\x03\x03`.
 - Need 8 bytes padding (full block) $\rightarrow$ add `\x08\x08\x08\x08\x08\x08\x08\x08`.
-

Slide 4.4: Block Cipher Modes - CFB and OFB

Cipher Feedback (CFB) Mode

Concept: Block cipher used as self-synchronizing stream cipher.

Encryption:

$$C_0 = IV$$

$$O_i = E(K, C_{i-1})$$

$$C_i = P_i \oplus O_i$$

Note: Decryption uses $E(K)$, not $D(K)$.

Output Feedback (OFB) Mode

Concept: Block cipher used as synchronous stream cipher. Keystream generated independently of plaintext/ciphertext.

Encryption/Decryption:

$$X_0 = IV$$

$$X_i = E(K, X_{i-1})$$

$$C_i = P_i \oplus X_i$$

Comparison: CFB vs OFB

Feature	CFB	OFB
Type	Self-synchronizing	Synchronous
Error propagation	Yes (limited)	No
Keystream	Depends on ciphertext	Independent
Precomputation	No	Yes (X_i can be precomputed)
Bit errors	Affects current + next block	Affects only current block

Slide 4.5: Feistel Cipher Structure

Definition

Feistel Cipher: Symmetric structure used in block cipher design (named after Horst Feistel). Used in DES, Blowfish, Twofish.

Key Insight: Allows decryption even if the round function F is not invertible.

Feistel Network Structure

Inputs:

- Plaintext block: $2w$ bits
- Key: K
- Number of rounds: n

Process:

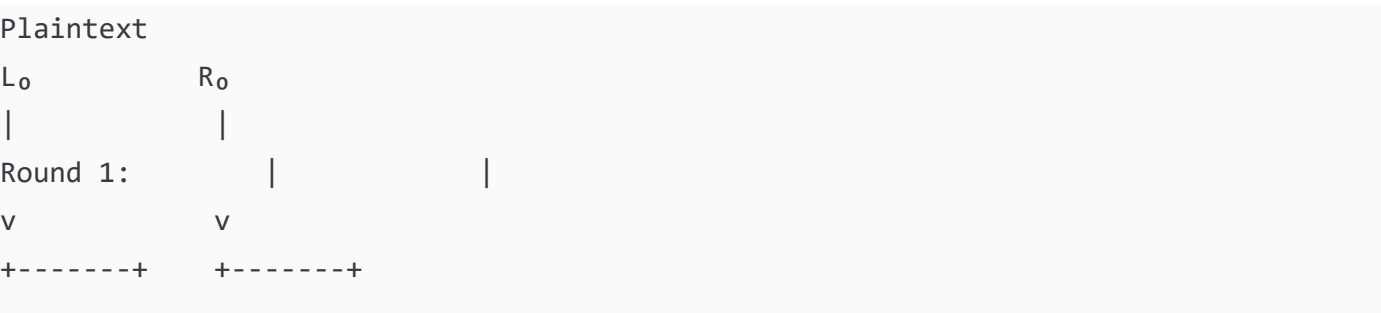
- Split plaintext into two halves: L_0, R_0 (w bits each).
- For each round $i = 1, 2, \dots, n$:

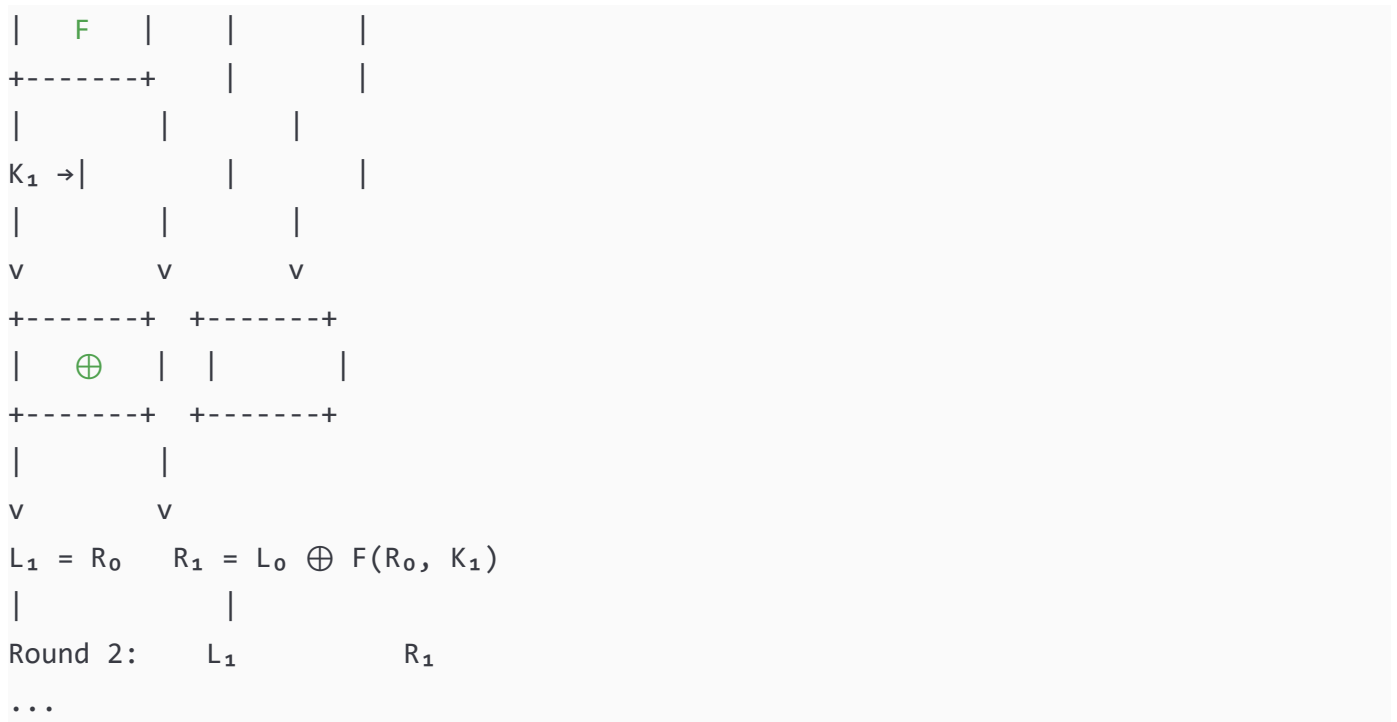
$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

- Ciphertext = $R_n || L_n$ (Note: swapped!)

Detailed Diagram (2 Rounds)





Decryption in Feistel Cipher

Key insight: Decryption uses same structure as encryption!

Simply apply rounds in **reverse order** with **reversed keys**.

For $i = n, n - 1, \dots, 1$:

$$R_{i-1} = L_i$$

$$L_{i-1} = R_i \oplus F(L_i, K_i)$$

Proof of Correctness (Round 1)

Encryption:

$$L_1 = R_0$$

$$R_1 = L_0 \oplus F(R_0, K_1)$$

Decryption (Input L_1, R_1):

1. Recover R_0 :

$$R_0 = L_1 \quad (\text{Direct from encryption})$$

2. Recover L_0 :

$$L_0 = R_1 \oplus F(L_1, K_1)$$

Substitute R_1 :

$$L_0 = [L_0 \oplus F(R_0, K_1)] \oplus F(L_1, K_1)$$

Since $L_1 = R_0$:

$$L_0 = L_0 \oplus F(R_0, K_1) \oplus F(R_0, K_1)$$

$$L_0 = L_0 \oplus 0$$

$$L_0 = L_0 \quad \checkmark$$

Advantages of Feistel Structure

- 1. $\checkmark$ **Invertibility guaranteed:** Even if F is not invertible (allows complex non-linear functions).
- 2. $\checkmark$ **Same hardware/software** for encryption and decryption (just reverse key schedule).
- 3. $\checkmark$ **Design flexibility:** Can use any function F .
- 4. $\checkmark$ **Proven security:** With enough rounds (e.g., 16 for DES).

WEEK 5: DES AND 3DES

Slide 5.1: DES (Data Encryption Standard)

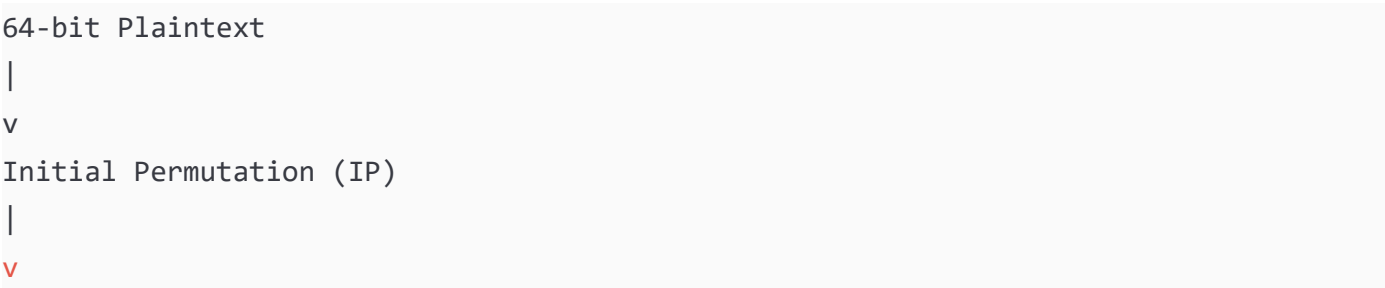
Historical Background

- **1972:** NBS (now NIST) requests encryption standard.
- **1974:** IBM submits Lucifer cipher.
- **1977:** DES adopted as FIPS 46.
- **1999:** DES broken in 22 hours (EFF Deep Crack).
- **2002:** Replaced by AES.

DES Parameters

Parameter	Value
Block size	64 bits
Key size	64 bits (56 effective + 8 parity)
Number of rounds	16
Structure	Feistel cipher
Subkey size	48 bits per round

Overall DES Structure



```

+-----+
|  16 Rounds  |  ← 56-bit key → 16 × 48-bit subkeys
+-----+
|
v
32-bit Swap ( $L_{16} \leftrightarrow R_{16}$ )
|
v
Final Permutation ( $IP^{-1}$ )
|
v
64-bit Ciphertext

```

Slide 5.2: DES - Initial and Final Permutation

Initial Permutation (IP)

Purpose: Rearrange bits (no cryptographic strength, likely for hardware ease in 1970s).

IP Table (read row by row):

```

58 50 42 34 26 18 10  2
60 52 44 36 28 20 12  4
...

```

Example:

- Input bit 58 → Output bit 1.
- Input bit 50 → Output bit 2.
- Input bit 2 → Output bit 8.

Final Permutation (IP^{-1})

Purpose: Inverse of IP.

Property: $IP^{-1}(IP(x)) = x$.

Slide 5.3: DES Round Function (The F Function)

One Round of DES

Input: L_{i-1} (32 bits), R_{i-1} (32 bits), K_i (48 bits)

Output: L_i (32 bits), R_i (32 bits)

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

The F Function Steps

1. **Expansion Permutation (E)**: 32 bits $\rightarrow$ 48 bits.
2. **XOR with Key**: 48 bits $\oplus$ 48 bits.
3. **S-Boxes**: 48 bits $\rightarrow$ 32 bits (Non-linear).
4. **Permutation (P)**: 32 bits $\rightarrow$ 32 bits.

1. Expansion Permutation (E)

Purpose: Expand 32 bits to 48 bits to match key size.

Mechanism: Some bits are duplicated.

Example:

- Bit 32 of R $\rightarrow$ Bit 1 of expanded.
- Bit 1 of R $\rightarrow$ Bit 2 of expanded.
- Bit 2 of R $\rightarrow$ Bit 3 of expanded.
- ...
- Bit 32 is repeated to provide diffusion.

2. S-Boxes (Substitution Boxes)

Purpose: Non-linear substitution (core of DES security).

Structure: 8 S-boxes, each takes 6 bits, outputs 4 bits.

S-Box Selection:

- **Row**: Bits 1 and 6 (Outer bits).
- **Column**: Bits 2,3,4,5 (Inner bits).

Detailed S-Box 1 Example

Input: 011011 (6 bits)

1. **Row**: Bits 1 & 6 = 01 (Binary) = 1 (Decimal).
2. **Column**: Bits 2-5 = 1101 (Binary) = 13 (Decimal).
3. **Lookup**: Go to Row 1, Column 13 of S-Box 1 table.
 - Table Value: 5.
4. **Output**: 5 = 0101 (4 bits).

3. Permutation (P)

P Table: Rearranges the 32 bits from S-Boxes to diffuse changes across the block.

Slide 5.4: DES Key Schedule

Key Schedule Overview

Input: 64-bit key (56 bits + 8 parity bits).

Output: 16 subkeys, each 48 bits.

Step 1: Permuted Choice 1 (PC-1)

Purpose: Remove parity bits (every 8th bit) and permute.

Result: 56 bits split into C_0 (28 bits) and D_0 (28 bits).

Step 2: Left Shifts

Shift schedule:

Round:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Shift:	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Example:

- $C_0 = c_1c_2\dots c_{28}$
- After 1-bit left shift: $C_1 = c_2c_3\dots c_{28}c_1$

Step 3: Permuted Choice 2 (PC-2)

Purpose: Select 48 bits from 56 bits ($C_i||D_i$).

Result: 48-bit Subkey K_i .

Example Key Schedule

Original key (hex): 133457799BBCDFF1

After PC-1:

- $C_0 = 1111000011001100101010101111$
- $D_0 = 01010101011001100111110001111$
- **Round 1 (shift 1):**
- $C_1 = 1110000110011001010101011111$
- $D_1 = 1010101011001100111100011110$
- K_1 (after PC-2) = 000110 110000 001011 101111...

Slide 5.5: DES Security Analysis

Key Space

Effective key size: 56 bits.

Total keys: $2^{56} \approx 7.2 \times 10^{16}$.

Brute force time:

- 1 key/ μ s $\rightarrow$ 2,285 years.
- Modern FPGA (10^9 keys/sec) $\rightarrow$ ~22 hours.

Attacks on DES

1. Brute Force Attack

- **1998:** EFF "Deep Crack" machine broke DES in **56 hours**.
- **1999:** Combined effort broke DES in **22 hours**.

2. Differential Cryptanalysis

- **Discovered:** Biham and Shamir (1990).
- **Idea:** Analyze how differences in plaintext affect differences in ciphertext.
- **Complexity:** 2^{47} chosen plaintexts.
- **DES resistance:** Designed to resist this (IBM knew about it in 1970s!).

3. Linear Cryptanalysis

- **Discovered:** Matsui (1993).
- **Idea:** Find linear approximations of DES rounds.
- **Complexity:** 2^{43} known plaintexts.

DES Weaknesses

1. ✗ **Small key size:** 56 bits insufficient for modern computing.
 2. ✗ **Small block size:** 64 bits vulnerable to birthday attacks on modes.
 3. ✗ **Slow in software:** Designed for 1970s hardware.
 4. ✓ **Resistant to cryptanalysis:** No major structural flaws found other than key size.
-

Slide 5.6: Triple DES (3DES)

Motivation

Problem: DES key size (56 bits) too small.

Solution: Apply DES multiple times with different keys.

3DES Variants

1. DES-EEE3 (Triple Encryption with 3 Keys)

Encryption: $C = E(K_3, E(K_2, E(K_1, P)))$

Key size: $3 \times 56 = 168$ bits.

Effective security: 112 bits (due to meet-in-the-middle attack).

2. DES-EDE3 (Encrypt-Decrypt-Encrypt with 3 Keys)

Encryption: $C = E(K_3, D(K_2, E(K_1, P)))$

Advantage: Backward compatible with DES (if $K_1 = K_2 = K_3$, then $E(D(E(P))) = E(P)$).

3. DES-EDE2 (Encrypt-Decrypt-Encrypt with 2 Keys)

Encryption: $C = E(K_1, D(K_2, E(K_1, P)))$

Key size: 112 bits.

Effective security: ~80 bits.

Meet-in-the-Middle Attack on Double DES

Double DES: $C = E(K_2, E(K_1, P))$

Naive security: 2^{112} (seems secure).

Attack:

1. For all K_1 : Compute $T = E(K_1, P)$, store (T, K_1) in table.
2. For all K_2 : Compute $T = D(K_2, C)$, look up in table.
3. Match found $\rightarrow$ candidate (K_1, K_2) .

Complexity:

- Time: $2^{56} + 2^{56} = 2^{57}$.
- Space: 2^{56} entries.

Result: Double DES only provides 57-bit security! (Not much better than DES).

Why 3DES works: Meet-in-the-middle on 3DES requires 2^{112} time, which is secure.

3DES Status

- **Advantages:** Well-studied, No practical attacks (on 3-key).
- **Disadvantages:** Slow (3× DES), Small block size (64 bits).
- **Status:** Deprecated by NIST (2023).
- **Usage:** Legacy systems, payment cards (EMV).

WEEK 6: IDEA AND AES

Slide 6.1: IDEA (International Data Encryption Algorithm)

Historical Background

- **Designed:** Lai and Massey (1991).
- **Used in:** PGP (Pretty Good Privacy).
- **Patent:** Expired in 2012.

IDEA Parameters

Parameter	Value
Block size	64 bits
Key size	128 bits

Parameter	Value
Number of rounds	8
Structure	Not Feistel (uses different design)

IDEA Operations

IDEA uses three group operations on 16-bit subblocks:

1. **XOR** ($\oplus$): Bitwise exclusive OR.
2. **Addition** ($\boxplus$): Addition modulo 2^{16} .
3. **Multiplication** ($\odot$): Multiplication modulo $2^{16} + 1$ (prime).

Multiplication Modulo $2^{16} + 1$

Special case: 0 represents 2^{16} (65536).

Example:

- $0x0000 \odot 0x1234 = 0x1234$ (0 represents 2^{16}).
- $0xFFFF \odot 0x0002 = (65535 \times 2) \bmod 65537 = 131070 \bmod 65537 = 1$.

IDEA Round Structure

Input: 64-bit plaintext split into 4×16 -bit blocks: X_1, X_2, X_3, X_4 .

Each round (1-8):

1. Multiply/Add with subkeys $Z_1..Z_4$.
2. Mix middle blocks (X_2, X_3) using Z_5, Z_6 .
3. Swap middle blocks.

Key Schedule

128-bit key $\rightarrow$ 52 subkeys (each 16 bits).

Generation:

- $K_1 - K_8$: First 128 bits (round 1).
- $K_9 - K_{16}$: Next 128 bits (round 2).
- ...
- **Rotation:** Key rotated 25 bits left for each round.

Slide 6.2: IDEA Security Analysis

Security Strengths

1. ✓ **No practical attacks:** Best attack is $2^{126.8}$ (essentially brute force).

2. ✓ **Resistant to differential cryptanalysis.**
3. ✓ **Resistant to linear cryptanalysis.**
4. ✓ **Efficient in software.**

IDEA vs DES

Feature	IDEA	DES
Block size	64 bits	64 bits
Key size	128 bits	56 bits
Structure	Non-Feistel	Feistel
Operations	$\oplus$, $\boxplus$, $\odot$	XOR, S-boxes
Speed (software)	Faster	Slower

Current Status

- **Advantages:** Still secure, Patent expired (free to use).
- **Disadvantages:** Complex (harder to implement securely), Less studied than AES.
- **Usage:** PGP, some legacy systems.

Slide 6.3: AES (Advanced Encryption Standard) - Overview

Historical Background

- **1997:** NIST calls for AES proposals.
- **2000:** Rijndael selected.
- **2001:** AES becomes FIPS 197.

AES Parameters

Parameter	AES-128	AES-192	AES-256
Block size	128 bits	128 bits	128 bits
Key size	128 bits	192 bits	256 bits
Number of rounds	10	12	14

AES Structure

Not a Feistel cipher! Uses Substitution-Permutation Network (SPN).

State: 4×4 matrix of bytes (128 bits).

State Matrix:

```
+-----+-----+-----+-----+
```

```

| a0 | a4 | a8 | a12 |
+-----+-----+-----+
| a1 | a5 | a9 | a13 |
+-----+-----+-----+
| a2 | a6 | a10 | a14 |
+-----+-----+-----+
| a3 | a7 | a11 | a15 |
+-----+-----+-----+

```

Note: Bytes are stored column-major in memory.

AES Round Structure

Each round (except final) consists of 4 transformations:

1. **SubBytes:** Non-linear substitution (S-box).
2. **ShiftRows:** Cyclic shift of rows.
3. **MixColumns:** Linear mixing of columns.
4. **AddRoundKey:** XOR with round key.

Final round: No MixColumns.

Slide 6.4: AES Transformations (Deep Dive)

1. SubBytes

Purpose: Non-linear byte substitution.

Process:

1. Take multiplicative inverse in $GF(2^8)$.
2. Apply affine transformation.

Example:

- Input: `0x53`
- Inverse in $GF(2^8)$: `0xCA`
- Affine transform: `0xED`
- S-box[`0x53`] = `0xED`

2. ShiftRows

Purpose: Diffusion across rows.

Process:

- Row 0: No shift.
- Row 1: Shift left by 1.
- Row 2: Shift left by 2.

- Row 3: Shift left by 3.

3. MixColumns (Mathematical Detail)

Purpose: Diffusion within columns.

Process: Each column treated as polynomial over $GF(2^8)$.

Matrix multiplication:

$$\begin{array}{c}
 +-----+ \quad +-----+-----+-----+-----+ \quad +-----+ \\
 | \ a_0 \ | \quad | \ 02 \ | \ 03 \ | \ 01 \ | \ 01 \ | \quad | \ a_0 \ | \\
 | \ a_1 \ | \ = \ | \ 01 \ | \ 02 \ | \ 03 \ | \ 01 \ | \ \times \ | \ a_1 \ | \\
 | \ a_2 \ | \quad | \ 01 \ | \ 01 \ | \ 02 \ | \ 03 \ | \quad | \ a_2 \ | \\
 | \ a_3 \ | \quad | \ 03 \ | \ 01 \ | \ 01 \ | \ 02 \ | \quad | \ a_3 \ | \\
 +-----+ \quad +-----+-----+-----+-----+ \quad +-----+
 \end{array}$$

Galois Field Multiplication:

- $02 \times a = (a \ll 1) \oplus (0x1B \text{ if } \text{MSB}(a) = 1)$
- $03 \times a = (02 \times a) \oplus a$

Detailed Calculation Example:

Column: $[0xDB, 0x13, 0x53, 0x45]$

Calculate New a_0 :

$$New_a_0 = (02 \odot 0xDB) \oplus (03 \odot 0x13) \oplus (01 \odot 0x53) \oplus (01 \odot 0x45)$$

- $02 \times 0xDB$ (11011011):
 - Shift left: 10110110 ($0xB6$).
 - MSB was 1, so XOR $0x1B$: $0xB6 \oplus 0x1B = 0xAD$.
- $03 \times 0x13$:
 - $02 \times 0x13 = 0x26$.
 - $0x26 \oplus 0x13 = 0x35$.
- $01 \times 0x53 = 0x53$.
- $01 \times 0x45 = 0x45$.
- XOR Sum: $0xAD \oplus 0x35 \oplus 0x53 \oplus 0x45$
 - $0xAD \oplus 0x35 = 0x98$
 - $0x98 \oplus 0x53 = 0xCB$
 - $0xCB \oplus 0x45 = 0x8E$

Result: $0x8E$.

4. AddRoundKey

Purpose: Introduce key material.

Process: Simple XOR with round key.

State: 0x8E 0xA9 0x2F 0xD3

RoundKey: 0x12 0x34 0x56 0x78

New State: 0x9C 0x9D 0x79 0xAB

Slide 6.5: AES Key Expansion

Purpose

Generate round keys from original key.

Key Expansion for AES-128

Input: 128-bit key (4 words: W_0, W_1, W_2, W_3).

Output: 44 words ($W_0, \dots, W_{43}$) for 10 rounds + initial.

Algorithm

```
def KeyExpansion(key):  
    W[0..3] = key # Split into 4 words  
    for i in range(4, 44):  
        temp = W[i-1]  
        if i % 4 == 0:  
            temp = SubWord(RotWord(temp)) ⊕ Rcon[i/4]  
        W[i] = W[i-4] ⊕ temp  
    return W
```

Operations

RotWord: Rotate 4-byte word left by 1 byte.

$[a_0, a_1, a_2, a_3] \rightarrow [a_1, a_2, a_3, a_0]$

SubWord: Apply S-box to each byte.

Rcon: Round constants ($0x01000000, 0x02000000, \dots$).

Example: First Few Words

Original key: 2B7E1516 28AED2A6 ABF71588 09CF4F3C

Calculate W_4 :

1. $W_3 = 09CF4F3C$
2. $\text{RotWord}(W_3) = \text{CF4F3C09}$
3. $\text{SubWord}(\text{CF4F3C09}) = \text{8B84EB01}$ (via S-box lookup)
4. $\text{Rcon}[1] = \text{01000000}$

5. $\text{Temp} = \boxed{8B84EB01} \oplus \boxed{01000000} = \boxed{8A84EB01}$

6. $W_4 = W_0 \oplus \text{Temp} = 2B7E1516 \oplus 8A84EB01 = A0FAFE17$

Slide 6.6: AES Security Analysis

Security Strengths

1. ✓ **No practical attacks** on full AES.
2. ✓ **Resistant to differential/linear cryptanalysis.**
3. ✓ **Efficient in hardware and software.**

Known Attacks

1. Biclique Attack (2011)

- **Complexity:** AES-128: $2^{126.1}$ (vs 2^{128} brute force).
- **Impact:** Marginal improvement, not practical.

2. Side-Channel Attacks

- **Types:** Timing, Power analysis, Cache attacks.
- **Mitigation:** Constant-time implementations, masking.

AES in Practice

Used in: TLS/SSL, WiFi (WPA2), Disk encryption, VPNs, Messaging.

Slide 6.7: Message Integrity

Definition

Message Integrity: Assurance that message has not been altered.

Why Confidentiality is Not Enough

Example: ECB mode encryption.

- Plaintext: "Transfer \$100 to Alice"
- Attacker modifies ciphertext bits.
- Decryption: "Transfer \$999 to Alice"
- **Problem:** Receiver has no way to detect modification!

Integrity Techniques

1. **Checksums:** Detect accidental errors (Not secure).
2. **CRC:** Detect accidental errors (Not secure).
3. **Cryptographic Hash Functions:** Detect intentional modifications (MD5, SHA-256).

WEEK 7: MESSAGE AUTHENTICATION AND HASH FUNCTIONS

Slide 7.1: Message Authentication Codes (MACs)

Definition

MAC: Short piece of information used to authenticate a message, generated using a secret key.

Mathematical Model

Let:

- **K** = Secret key
- **M** = Message
- **MAC** = Authentication tag

MAC Generation: $T = MAC(K, M)$

Verification: Accept if $T' = MAC(K, M')$

Security Requirements

1. **Unforgeability:** Without K, cannot generate valid MAC.
 2. **Computational resistance:** Given (M, T) , cannot find (M', T') .
 3. **Key non-recovery:** Cannot recover K from (M, T) pairs.
-

Slide 7.2: MAC Construction Methods

1. CBC-MAC

Concept: Use block cipher in CBC mode, take last block as MAC.

Algorithm:

Input: $M = M_1 || M_2 || \dots || M_n$, Key K

$IV = \emptyset$

$C_0 = IV$

For $i = 1$ to n :

$C_i = E(K, M_i \oplus C_{i-1})$

MAC = C_n

Problem: Insecure for variable-length messages!

Attack:

- Given: $MAC(M) = T$
- Construct: $M' = M || (M_1 \oplus T) || M_2 \dots$
- Then: $MAC(M') = T$

2. CMAC (Cipher-based MAC)

Solution: Fix CBC-MAC for variable-length messages.

Algorithm:

- If M is complete blocks: XOR last block with K_1 , then encrypt.
- Else: Pad last block, XOR with K_2 , then encrypt.

Standard: NIST SP 800-38B.

3. HMAC (Hash-based MAC)

Concept: Use cryptographic hash function with key.

Algorithm:

$$HMAC(K, M) = H((K' \oplus opad) || H((K' \oplus ipad) || M))$$

where:

- H = Hash function (SHA-256).
- K' = Key padded to block size.
- $ipad = 0x363636\dots$
- $opad = 0x5C5C5C\dots$

HMAC Security

Advantages: Proven secure (if H is secure), Fast, Widely implemented.

Used in: TLS, IPsec, SSH, JWT.

Slide 7.3: Universal Hashing

Definition

Universal Hashing: Family of hash functions where probability of collision is low.

Mathematical Definition

Let H be a family of hash functions $h : U \rightarrow \{0, 1, \dots, m - 1\}$.

H is **universal** if for any distinct $x, y \in U$:

$$Pr_{h \in H}[h(x) = h(y)] \leq \frac{1}{m}$$

Example: Universal Hash Family

Construction:

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$$

where $p = \text{prime} > |U|$.

Application: Message Authentication

Wegman-Carter MAC:

$$MAC(K_1, K_2, M) = h_{K_1}(M) \oplus K_2$$

Security: Information-theoretically secure (if K_2 truly random).

Slide 7.4: MD5 Hash Function

Historical Background

- **Designed:** Ron Rivest (1991).
- **Output:** 128 bits.
- **Status:** X **BROKEN** (do not use!).

MD5 Vulnerabilities

1. Collision Attacks

- **2004:** Wang et al. found collisions in hours.
- **2008:** Collisions found in seconds.
- **Example:** Two different X.509 certificates with same MD5 hash.

2. Chosen-Prefix Collisions

- **2012:** Flame malware used MD5 collision to forge certificate.

Current Status

X **Do NOT use for:** Digital signatures, Certificates, Password hashing.

~ **OK for** (non-security): Checksums (detect accidental corruption).

Slide 7.5: SHA Family

SHA-1

- **Output:** 160 bits.
- **Status:** X **BROKEN** (deprecated).
- **2017:** Google's "SHAttered" attack (First practical collision).

SHA-2 Family

- **Variants:** SHA-224, SHA-256, SHA-384, SHA-512.
- **Structure:** Merkle-Damgård construction.

SHA-256 Algorithm

Input: Message $< 2^{64}$ bits.

Output: 256-bit hash.

Steps:

1. **Preprocessing:** Pad message to multiple of 512 bits. Append length (64 bits).
2. **Initialize hash values:** 8×32 -bit constants ($H_0 \dots H_7$).
3. **Process each 512-bit block:** Expand to 64 words, 64 rounds of compression.
4. **Output:** $H_0 || H_1 || \dots || H_7$.

SHA-256 Compression Function

Each round:

$$T_1 = h + \Sigma_1(e) + Ch(e, f, g) + K_t + W_t$$

$$T_2 = \Sigma_0(a) + Maj(a, b, c)$$

$$e = d + T_1$$

$$a = T_1 + T_2$$

(Plus rotations of other variables).

SHA-3 (Keccak)

- **Selected:** NIST competition (2012).
- **Different from SHA-2:** Sponge construction (not Merkle-Damgård).
- **Advantages:** Resistant to length extension attacks.

Slide 7.6: Collision-Resistant Hash Functions

Definition

Collision-Resistant Hash Function (CRHF): Hash function where it's computationally infeasible to find two distinct inputs with same output.

Security Properties

1. **Preimage Resistance (One-Way):** Given h , find m such that $H(m) = h$. Complexity: $O(2^n)$.
2. **Second Preimage Resistance:** Given m_1 , find m_2 such that $H(m_1) = H(m_2)$. Complexity: $O(2^n)$.

3. **Collision Resistance:** Find any m_1, m_2 such that $H(m_1) = H(m_2)$. Complexity: $O(2^{n/2})$ (Birthday Attack).

Birthday Attack

Birthday Paradox: In a group of 23 people, probability of shared birthday > 50%.

General formula: For n-bit hash, need $\approx 2^{n/2}$ trials to find collision.

Detailed Example: Birthday Attack on MD5

MD5 output: 128 bits.

Brute force collision: $2^{128} \approx 3.4 \times 10^{38}$.

Birthday attack: $2^{64} \approx 1.8 \times 10^{19}$.

Time (10^9 hashes/sec):

$$\frac{2^{64}}{10^9} \approx 585 \text{ years}$$

But with optimizations (Differential cryptanalysis): Hours/days!

Hash Function Security Levels

Hash	Output	Collision Security	Status
MD5	128 bits	64 bits	X Broken
SHA-1	160 bits	80 bits	X Broken
SHA-256	256 bits	128 bits	✓ Secure
SHA-512	512 bits	256 bits	✓ Secure

WEEK 8: ADVANCED TOPICS IN HASH FUNCTIONS

Slide 8.1: Birthday Attack - Detailed Analysis

Mathematical Foundation

Birthday Problem: What's the probability that in a group of k people, at least two share a birthday?

Assumptions: 365 days, uniform distribution.

Probability Calculation

Probability of NO collision:

$$P(\text{no collision}) = \frac{365}{365} \times \frac{364}{365} \times \dots \times \frac{365 - k + 1}{365}$$

Probability of AT LEAST ONE collision:

$$P(\text{collision}) = 1 - P(\text{no collision})$$

Example Calculations

k (people)	P(collision)
10	11.7%
23	50.7%
50	97.0%
70	99.9%

Generalization to Hash Functions

For n-bit hash function:

- Number of possible outputs: $N = 2^n$.
- Number of hashes needed for 50% collision probability: $k \approx 1.177\sqrt{N}$.

Formula:

$$k \approx \sqrt{2 \cdot 2^n \cdot \ln(2)} \approx 1.177 \cdot 2^{n/2}$$

Real-World Birthday Attacks

1. **MD5 Collision (2004)**: Complexity 2^{39} (much better than 2^{64}).
2. **SHA-1 Collision (2017)**: Google's "SHAttered". Complexity $2^{63.1}$. Cost ~\$110,000.

Defense Against Birthday Attacks

1. **Use longer hash outputs**: ≥ 256 bits.
2. **Randomized hashing**: Add random salt.
3. **Avoid deprecated hashes**: No MD5, SHA-1.

Slide 8.2: Authenticated Encryption

Problem Statement

Traditional approach:

- Encrypt for confidentiality ($C = E(K_e, M)$).
- Add MAC for integrity ($T = MAC(K_m, M)$).
- Send (C, T) .

Problems: Two keys, Two passes, Composition issues.

Authenticated Encryption (AE)

Definition: Single primitive providing both confidentiality and integrity.

Output: Ciphertext + authentication tag.

Notation: $(C, T) = AE(K, M)$.

Security Goals

1. **Confidentiality:** IND-CPA.
2. **Integrity:** INT-CTXT.
3. **Combined:** IND-CCA2.

Composition Methods

1. Encrypt-and-MAC

- $C = E(K_e, M), T = MAC(K_m, M)$.
- **Problem:** MAC may leak information about M.

2. MAC-then-Encrypt

- $T = MAC(K_m, M), C = E(K_e, M || T)$.
- **Problem:** Vulnerable to padding oracle attacks.

3. Encrypt-then-MAC ✓ RECOMMENDED

- $C = E(K_e, M), T = MAC(K_m, C)$.
 - **Advantages:** Provably secure, MAC protects ciphertext.
 - **Used in:** IPsec, TLS 1.3.
-

Slide 8.3: Authenticated Encryption Modes

1. GCM (Galois/Counter Mode)

Structure: CTR mode + GMAC (Galois MAC).

Advantages: Parallelizable, Fast in hardware, Widely used (TLS, SSH).

Algorithm:

1. Encryption: $C = \text{CTR_Encrypt}(K, IV, M)$.
2. Tag: $T = \text{GMAC}(K, C, A)$ (A = associated data).
3. Output: (C, T) .

2. CCM (Counter with CBC-MAC)

Structure: CBC-MAC + CTR mode.

Used in: WiFi (WPA2), Bluetooth.

Disadvantages: Not parallelizable.

3. OCB (Offset Codebook Mode)

Advantages: Fastest AE mode, Single pass.

Disadvantages: Patent issues (now expired).

Slide 8.4: Euler's Totient Function

Definition

Euler's Totient Function $\phi(n)$: Count of positive integers $\leq n$ that are coprime to n .

Mathematical Definition

$$\phi(n) = |\{k \in \mathbb{Z} : 1 \leq k \leq n, \gcd(k, n) = 1\}|$$

Examples

- $\phi(1) = 1$
- $\phi(5) = 4 \{1, 2, 3, 4\}$
- $\phi(10) = 4 \{1, 3, 7, 9\}$

Properties

1. For Prime p

$$\phi(p) = p - 1$$

2. For Prime Powers

$$\phi(p^k) = p^k - p^{k-1} = p^k \left(1 - \frac{1}{p}\right)$$

Example: $\phi(8) = \phi(2^3) = 8 - 4 = 4$.

3. Multiplicative Property

If $\gcd(m, n) = 1$, then:

$$\phi(mn) = \phi(m) \cdot \phi(n)$$

Example: $\phi(15) = \phi(3) \cdot \phi(5) = 2 \cdot 4 = 8$.

General Formula

For $n = p_1^{k_1} \cdot p_2^{k_2} \cdots$:

$$\phi(n) = n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdots$$

Example Calculations

$\phi(12)$:

$$12 = 2^2 \cdot 3$$

$$\phi(12) = 12 \cdot (1 - 1/2) \cdot (1 - 1/3) = 12 \cdot 1/2 \cdot 2/3 = 4$$

Verify: $\{1, 5, 7, 11\} = 4$ numbers $\checkmark$

$\phi(100)$:

$$100 = 2^2 \cdot 5^2$$

$$\phi(100) = 100 \cdot 1/2 \cdot 4/5 = 40$$

Slide 8.5: Euler's Theorem

Statement

If $\gcd(a, n) = 1$, then:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

Proof

Let $R = \{r_1, \dots, r_{\phi(n)}\}$ be integers coprime to n .

Consider $S = \{a \cdot r_1 \pmod{n}, \dots, a \cdot r_{\phi(n)} \pmod{n}\}$.

Claim: $S = R$ (as sets).

Proof:

1. Each $a \cdot r_i$ is coprime to n .

2. All $a \cdot r_i$ are distinct.

Therefore:

$$\prod (a \cdot r_i) \equiv \prod r_i \pmod{n}$$

$$a^{\phi(n)} \cdot \prod r_i \equiv \prod r_i \pmod{n}$$

Cancel $\prod r_i$:

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad \checkmark$$

Examples

Example 1: $a = 3, n = 10$.

$$\phi(10) = 4.$$

$$3^4 = 81 \equiv 1 \pmod{10} \quad \checkmark$$

Example 2: $a = 7, n = 12$.

$$\phi(12) = 4.$$

$$7^4 = 2401 = 200 \times 12 + 1 \equiv 1 \pmod{12} \quad \checkmark$$

Application: Modular Exponentiation

Problem: Compute $a^b \bmod n$ efficiently.

Solution: Use Euler's theorem to reduce exponent.

If $\gcd(a, n) = 1$:

$$a^b \bmod n = a^{b \bmod \phi(n)} \bmod n$$

Example: $3^{100} \bmod 10$.

$\phi(10) = 4$.

$100 \bmod 4 = 0$.

$3^{100} \bmod 10 = 3^0 \bmod 10 = 1$.

WEEK 9: PUBLIC KEY CRYPTOGRAPHY

Slide 9.1: Introduction to Public Key Cryptography

Problem with Symmetric Cryptography

Key Distribution Problem:

- Alice and Bob need shared secret key K .
- How to share K securely?
- **Key Management:** n users need $n(n-1)/2$ keys.
 - 100 users $\rightarrow$ 4,950 keys.
 - 1000 users $\rightarrow$ 499,500 keys.

Public Key Cryptography Solution

Idea: Each user has TWO keys:

1. **Public Key (PK):** Published, known to everyone.
2. **Private Key (SK):** Kept secret.

Diagram: Public Key System



```
|  Send C -----> |
|
|          Decrypt with SK_Bob
|          M = D(SK_Bob, C)
```

Properties

1. **Easy:** Generate key pair (PK, SK).
2. **Easy:** Encrypt with PK.
3. **Easy:** Decrypt with SK.
4. **Hard:** Derive SK from PK.
5. **Hard:** Decrypt without SK.

Applications

1. **Confidentiality:** Encrypt with recipient's public key.
2. **Authentication:** Sign with sender's private key.
3. **Key Exchange:** Establish shared secret (Diffie-Hellman).
4. **Digital Signatures:** Prove authenticity.

Slide 9.2: RSA Algorithm - Key Generation

Historical Background

- **Inventors:** Rivest, Shamir, Adleman (1977).
- **Based on:** Difficulty of factoring large integers.

RSA Key Generation (Step-by-Step)

Step 1: Choose two large primes p and q .

Example: $p = 61, q = 53$.

Step 2: Compute $n = p \times q$.

$$n = 61 \times 53 = 3233$$

Step 3: Compute $\phi(n) = (p - 1)(q - 1)$.

$$\phi(3233) = 60 \times 52 = 3120$$

Step 4: Choose e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.

Example: $e = 17$ (prime, doesn't divide 3120).

Step 5: Compute d such that $d \cdot e \equiv 1 \pmod{\phi(n)}$.

Use Extended Euclidean Algorithm:

$$d \cdot 17 \equiv 1 \pmod{3120}$$

$$d = 2753$$

Verification:

$$17 \times 2753 = 46801 = 15 \times 3120 + 1 \quad \checkmark$$

Step 6: Public and Private Keys.

Public Key: $(n, e) = (3233, 17)$.

Private Key: $(n, d) = (3233, 2753)$.

Slide 9.3: RSA Encryption and Decryption

Encryption

Given: Public key (n, e) , Plaintext M (integer, $0 \leq M < n$).

Compute:

$$C = M^e \pmod n$$

Decryption

Given: Private key (n, d) , Ciphertext C .

Compute:

$$M = C^d \pmod n$$

Correctness Proof

We need to show: $(M^e)^d \equiv M \pmod n$.

Since $d \cdot e \equiv 1 \pmod{\phi(n)}$:

$$d \cdot e = 1 + k \cdot \phi(n)$$

$$(M^e)^d = M^{ed} = M^{1+k\phi(n)} = M \cdot (M^{\phi(n)})^k$$

By Euler's theorem (if $\gcd(M, n) = 1$):

$$M^{\phi(n)} \equiv 1 \pmod n$$

Therefore:

$$M \cdot (1)^k \equiv M \pmod n \quad \checkmark$$

Detailed Example

Keys: Public: $(3233, 17)$, Private: $(3233, 2753)$.

Encryption:

Plaintext: $M = 65$.

$$C = 65^{17} \pmod{3233}$$

Using modular exponentiation (Square-and-Multiply):

- $17 = 10001_2$
 - $65^1 = 65$
 - $65^2 = 4225 \bmod 3233 = 992$
 - $65^4 = 992^2 \bmod 3233 = 2809$
 - $65^8 = 2809^2 \bmod 3233 = 2642$
 - $65^{16} = 2642^2 \bmod 3233 = 2581$
 - $65^{17} = 65^{16} \times 65^1 = 2581 \times 65 \bmod 3233 = 167765 \bmod 3233 = 2790$.
- Ciphertext:** 2790.

Decryption:

$$M = 2790^{2753} \bmod 3233$$

(Using similar modular exponentiation steps)

Result: 65 ✓

Practical RSA

Key sizes:

- **1024 bits:** Minimum (deprecated).
- **2048 bits:** Recommended (as of 2026).
- **3072 bits:** Long-term security.

Padding: Never use "textbook RSA"!

OAEP (Optimal Asymmetric Encryption Padding): Adds randomness, prevents attacks.

Slide 9.4: RSA Security Analysis

Security Assumption

RSA Problem: Given (n, e, C) , find M such that $C \equiv M^e \pmod{n}$.

Believed hard if n is large (≥ 2048 bits).

Attacks on RSA

1. Factoring Attack

Idea: Factor $n = p \times q$, then compute $\phi(n)$ and d .

Best known algorithm: General Number Field Sieve (GNFS).

Records:

- 2009: RSA-768 (768 bits).

- 2026: 1024 bits feasible for nation-states.

Recommendation: Use ≥ 2048 bits.

2. Small Exponent Attack

Scenario: $e = 3$ (small public exponent).

Attack: If $M^e < n$, then $C = M^e$ (no modulo).

Simply compute: $M = \sqrt[e]{C}$.

Defense: Use padding (OAEP).

3. Common Modulus Attack

Scenario: Same n , different e_1, e_2 .

Given $C_1 = M^{e_1} \bmod n$, $C_2 = M^{e_2} \bmod n$.

If $\gcd(e_1, e_2) = 1$, use Extended Euclidean Algorithm:

$$a \cdot e_1 + b \cdot e_2 = 1$$

Then:

$$M = C_1^a \cdot C_2^b \bmod n$$

Defense: Never reuse n .

4. Timing Attack

Idea: Measure decryption time to recover d .

Defense: Constant-time implementations.

5. Chosen Ciphertext Attack

Bleichenbacher's Attack (1998):

Scenario: PKCS#1 v1.5 padding.

Attack: Send modified ciphertexts, observe if padding valid.

Recover plaintext with $\sim 2^{20}$ queries.

Defense: Use OAEP padding, constant-time verification.

RSA Best Practices

✓ Use $n \geq 2048$ bits.

✓ Use $e = 65537$ ($2^{16} + 1$).

✓ Use OAEP padding.

✓ Constant-time implementation.

✗ Never use textbook RSA.

Slide 9.5: Pretty Good Privacy (PGP)

Historical Background

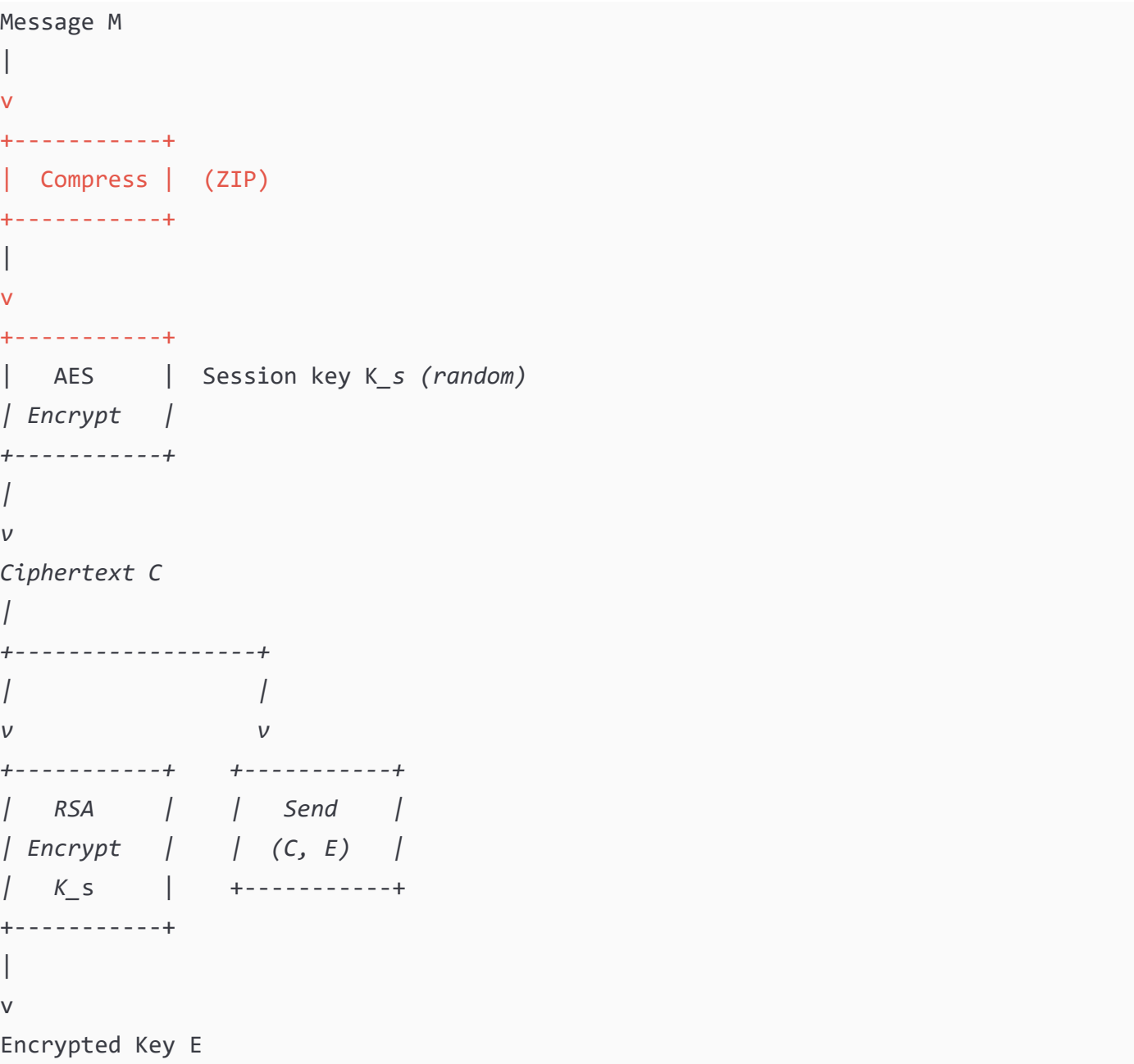
- **Created:** Phil Zimmermann (1991).
- **Purpose:** Email encryption for everyone.
- **Standard:** OpenPGP (RFC 4880).

PGP Hybrid Cryptosystem

Idea: Combine symmetric and asymmetric crypto.

- Asymmetric (RSA): Slow, but solves key distribution.
 - Symmetric (AES): Fast, but needs shared key.
- Solution:** Use asymmetric to encrypt symmetric key.

PGP Encryption Process



Detailed Steps

Sender:

1. Generate random session key K_s .
2. Compress message: $M' = \text{ZIP}(M)$.
3. Encrypt with K_s : $C = \text{AES}(K_s, M')$.
4. Encrypt K_s with recipient's public key: $E = \text{RSA}(PK_{\text{recipient}}, K_s)$.
5. Send (C, E) .

Receiver:

1. Decrypt session key: $K_s = \text{RSA}(SK_{\text{recipient}}, E)$.
2. Decrypt message: $M' = \text{AES}(K_s, C)$.
3. Decompress: $M = \text{UNZIP}(M')$.

PGP Digital Signature

Purpose: Authentication and integrity.

Process:

1. Compute $H = \text{SHA-256}(M)$.
2. Sign H with sender's private key: $S = \text{RSA}(SK_{\text{sender}}, H)$.
3. Send (M, S) .

Verification:

4. Compute $H' = \text{SHA-256}(M)$.
5. Verify signature: $H = \text{RSA}(PK_{\text{sender}}, S)$.
6. Check $H' = H$.

Web of Trust

Problem: How to trust public keys?

Solution: Decentralized trust model.

- Users sign each other's keys.
- Trust is transitive.
- No central authority (unlike PKI).

PGP vs S/MIME

Feature	PGP/OpenPGP	S/MIME
Trust Model	Web of Trust	PKI (Certificate Authorities)
Standard	Open (RFC 4880)	Proprietary (RSA)
Cost	Free (GPG)	Often requires paid certs
Adoption	Tech-savvy users	Corporate email

Modern PGP Tools

GnuPG (GPG):

- Open-source implementation.
- Command-line and GUI.
- Cross-platform.

Usage:

```
# Generate key
gpg --gen-key
# Export public key
gpg --armor --export alice@example.com
# Encrypt file
gpg --encrypt --recipient bob@example.com file.txt
# Sign file
gpg --sign file.txt
# Verify signature
gpg --verify file.txt.sig
```

PGP Limitations

- X **Usability**: Complex for average users.
- X **Key management**: Lost keys = lost data.
- X **No forward secrecy**: Compromised key decrypts all past messages.
- X **Metadata not encrypted**: Subject, To, From visible.

Modern Alternatives

Signal Protocol:

- End-to-end encryption.
- Forward secrecy.
- Easier to use.
- Used in WhatsApp, Signal.

ProtonMail:

- PGP-based email.
- Easier key management.
- Web interface.

APPENDIX: MATHEMATICAL REFERENCE

A.1: Modular Arithmetic

Definition

$a \equiv b \pmod{n}$ if n divides $(a - b)$.

Properties

1. **Reflexive:** $a \equiv a \pmod{n}$.
2. **Symmetric:** If $a \equiv b \pmod{n}$, then $b \equiv a \pmod{n}$.
3. **Transitive:** If $a \equiv b$ and $b \equiv c \pmod{n}$, then $a \equiv c \pmod{n}$.
4. **Addition:** If $a \equiv b$ and $c \equiv d \pmod{n}$, then $a + c \equiv b + d \pmod{n}$.
5. **Multiplication:** If $a \equiv b$ and $c \equiv d \pmod{n}$, then $ac \equiv bd \pmod{n}$.

Modular Inverse

Definition: $a^{-1} \pmod{n}$ exists if $\gcd(a, n) = 1$.

Computation: Extended Euclidean Algorithm.

Detailed Example: Find $7^{-1} \pmod{26}$

```
26 = 7×3 + 5
7 = 5×1 + 2
5 = 2×2 + 1
2 = 1×2 + 0
Back-substitute:
1 = 5 - 2×2
1 = 5 - (7 - 5×1)×2
1 = 5×3 - 7×2
1 = (26 - 7×3)×3 - 7×2
1 = 26×3 - 7×11
Therefore: 7×(-11) ≡ 1 (mod 26)
7×15 ≡ 1 (mod 26) (Since -11 mod 26 = 15)
7-1 mod 26 = 15
```

A.2: Finite Fields (Galois Fields)

GF(p) - Prime Field

Elements: $\{0, 1, 2, \dots, p - 1\}$.

Operations: Addition and multiplication mod p .

Example: GF(7)

- Addition: $5 + 4 = 9 \pmod{7} = 2$.
- Multiplication: $5 \times 4 = 20 \pmod{7} = 6$.

GF(2⁸) - Binary Field

Used in: AES, CRC.

Elements: Polynomials of degree < 8 with coefficients in {0, 1}.

Representation: 8-bit bytes.

Addition: XOR.

Multiplication: Polynomial multiplication mod irreducible polynomial.

AES irreducible polynomial: $m(x) = x^8 + x^4 + x^3 + x + 1$.

Detailed Example: $0x57 \times 0x83$ in GF(2⁸)

$0x57 = 01010111 = x^6 + x^4 + x^2 + x + 1$

$0x83 = 10000011 = x^7 + x + 1$

Multiply:

$(x^6 + x^4 + x^2 + x + 1)(x^7 + x + 1)$

$= x^{13} + x^{11} + x^9 + x + x^7 + x + x^5 + x^3 + x^2 + x + x^6 + x^4 + x^2 + x + 1$

$= x^{13} + x^{11} + x^9 + x^7 + x^6 + x^5 + x^4 + x^3 + 1$ (Simplify coefficients mod 2)

Reduce mod $m(x)$:

Result: $0xC1$

A.3: Chinese Remainder Theorem

Statement

Given:

- $x \equiv a_1 \pmod{n_1}$
- $x \equiv a_2 \pmod{n_2}$
- ...
- $x \equiv a_k \pmod{n_k}$

where $n_1, n_2, \dots, n_k$ are pairwise coprime.

Then there exists a unique solution modulo $N = n_1 n_2 \dots n_k$.

Solution Method

$$x = \sum_{i=1}^k a_i \cdot N_i \cdot M_i \pmod{N}$$

where:

- $N = n_1 n_2 \dots n_k$
- $N_i = N / n_i$
- $M_i = N_i^{-1} \pmod{n_i}$

Detailed Example

Solve:

- $x \equiv 2 \pmod{3}$
- $x \equiv 3 \pmod{5}$
- $x \equiv 2 \pmod{7}$

Solution:

- $N = 3 \times 5 \times 7 = 105$.
- $N_1 = 105/3 = 35$. $M_1 = 35^{-1} \pmod{3} = 2^{-1} \pmod{3} = 2$.
- $N_2 = 105/5 = 21$. $M_2 = 21^{-1} \pmod{5} = 1^{-1} \pmod{5} = 1$.
- $N_3 = 105/7 = 15$. $M_3 = 15^{-1} \pmod{7} = 1^{-1} \pmod{7} = 1$.

$$x = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 \pmod{105}$$

$$x = 140 + 63 + 30 \pmod{105}$$

$$x = 233 \pmod{105} = 23$$

Verify:

- $23 \pmod{3} = 2 \checkmark$
- $23 \pmod{5} = 3 \checkmark$
- $23 \pmod{7} = 2 \checkmark$

Application to RSA

CRT optimization: Speed up RSA decryption.

Instead of: $M = C^d \pmod{n}$.

Compute:

- $M_1 = C^d \pmod{p}$
- $M_2 = C^d \pmod{q}$
- Combine using CRT.

Speedup: $\sim 4\times$ faster.